Workshop on Computational Fluid Dynamics (CFD)
with OpenFOAM
IMP-PAN Gdańsk

Tutorial #3: Compressible air Ejector

Supersonic Transient and Steady Flow

Robert Castilla
CATMech - Universitat Politècnica de Catalunya
Department of Fluid Mechanics

2025-26
Version 1.0

In this tutorial the simulation of high velocity compressible flow is covered with the study of a supersonic ejector. Two density-based solvers are used: an explicit transient included in the OpenFOAM distribution, and an implicit steady-state, freely available in Internet.

Contents

Learning Objectives

By the end of this tutorial the student will be able to

1. **Understand** the basics differences between a pressure-based solver and a density-based solver
2. **Generate** a 2D axi-symmetric mesh for supersonic air ejector
3. **Setup and run** a simulation with an explicit transient solver and a steady-state implicit solver
4. **Compare** performance and results from explicit and implicit solvers

1 Introduction and Theoretical Background

In this session a supersonic air ejector is simulated. In this case high velocity air is used to create a very low pressure region with a shock wave. After this first shock wave secondary flow air is entrained and mixed with the main jet. This mixed flow is accelerated in the mixing chamber until a second shock wave is created in the diffuser.

This simulation is challenging because of the high Ma number compressible flow. For the previous sessions, where fluid were either liquids of low velocity gas, the simulations where performed by using a **pressure-based solver** (SIMPLE, PISO or PIMPLE algorithm). Momentum balance is computed by defining an equation for pressure (Poisson equation) that is corrected with continuity and then again computed with momentum equation until convergence. This method is not valid for high Ma number flows, since is not capable to adequately resolve shock waves. Instead, a **density-based solver** is used. These solvers consider energy equation, computing pressure from density and perfect gas equation (or any other gas state equation). From a mathematic point of view, in the pressure-based solvers, sound is considered to move instantaneously and the system of equations is of elliptic type (Poisson equation). But if compressibility of air is considered, the sound travels with finite speed and the system of equations is hyperbolic. The way they are solved is completely different.

The unsteady compressible flow equations for mass, momentum, and energy are

$$\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_i}\rho u_i = 0 \tag{1}$$

$$\frac{\partial}{\partial t}\rho u_j + \frac{\partial}{\partial x_i}\rho u_i u_j = -\frac{\partial p}{\partial x_i} + \frac{\partial}{\partial x_i}\sigma_{ij} \tag{2}$$

$$\frac{\partial}{\partial t}\rho E + \frac{\partial}{\partial x_i}\rho E u_i = -\frac{\partial u_i p}{\partial x_i} + \frac{\partial}{\partial x_i}\left(u_i \sigma_{ij} + k\frac{\partial T}{\partial x_j}\right) \tag{3}$$

along with the equation for perfect gases

$$p = \rho R' T \tag{4}$$

where $E = c_v T + \frac{1}{2}|\mathbf{u}|^2$, c_v is the specific heat at constant volume and R' is the constant for the gas.

Equations (1)–(3) can be expressed as

$$\frac{\partial \mathbf{W}}{\partial t} = \frac{\partial \mathbf{F_v}}{\partial x_i} - \frac{\partial \mathbf{F_c}}{\partial x_i} \tag{5}$$

where

$$\mathbf{W} = [\rho, \rho u_i, \rho E] \tag{6}$$

$$\mathbf{F_v} = [0, \sigma_{ij}, u_i \sigma_{ij} + k\frac{\partial T}{\partial x_j}] \tag{7}$$

$$\mathbf{F_c} = [\rho u_i, \rho u_i u_j + p\delta_{ij}, \rho E u_i + p u_i \delta_{ij}] \tag{8}$$

with

$$\sigma_{ij} = \mu\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} - \frac{2}{3}\frac{\partial u_k}{\partial x_k}\delta_{ij}\right) \tag{9}$$

An explicit and an implicit solver are used for comparison purposes. The explicit solver, `rhoCentralFoam`, included in the standard OpenFOAM distribution, is a transient density-based solver that employs the Kurganov–Tadmor central-upwind scheme for face interpolation. If a first-order Euler explicit time integration scheme is applied, the semi-discrete form of the governing equations is:

$$V\frac{\mathbf{W}^{n+1} - \mathbf{W}^n}{\Delta t} = R(\mathbf{W}^n) \tag{10}$$

where V is the cell volume and $R(\mathbf{W})$ is the right side of Equation (5) after face integration. This integration is typically performed with a flux limiter; in the present case, a van Leer limiter was used. The Courant–Friedrichs–Lewy (CFL) number is typically set ≤ 1 for stability.

The open source transient implicit solver used is **HiSA** by CSIR, which implements the Advection Upstream Splitting Method $(\text{AUSM})^{+\text{up}}$ upwind scheme for face flux computation. This solver enables the simulation of unsteady flows with significantly larger CFL numbers than explicit solvers. Moreover, several studies have reported that implicit schemes offer greater accuracy and efficiency at high Mach numbers. In the present case, however, a steady-state time scheme is used, reducing the governing equations to the non-linear residual form:

$$R(\mathbf{W}^{n+1}) = 0 \tag{11}$$

These equations are solved using the Generalized Minimal Residual (GMRES) method, with Lower–Upper Symmetric Gauss–Seidel (LU-SGS) preconditioning.

The turbulence model selected is the k-ω SST, as it has been shown to yield improved accuracy for this type of flow when compared with k-ϵ , k-ϵ realizable and stress-ω Reynolds Stress Models.

The implicit approach is more stable and robust, but, on the other hand, it requires a more complex solver configuration and is more computational resources demanding, both in time and memory[1].

1.1 Case Overview

The scheme of the ejector is depicted in Figure 1. Note that the primary nozzle has now the typical convergence-divergent shape in order to speed up the primary flow and get a shock wave. Also, the outlet has been enlarged in order to avoid as much as possible pressure wave reflections.

After the primary nozzle there is a convergent part that accelerates the secondary entrained flow, leading it to the mixing chamber, where both jets transfer momentum and energy. Before the outlet to the atmosphere the diffuser recovers the pressure after the second shock wave.

[1] For detailed information on numerical methodology the paper by Llorenç Macià et al. and references therein.

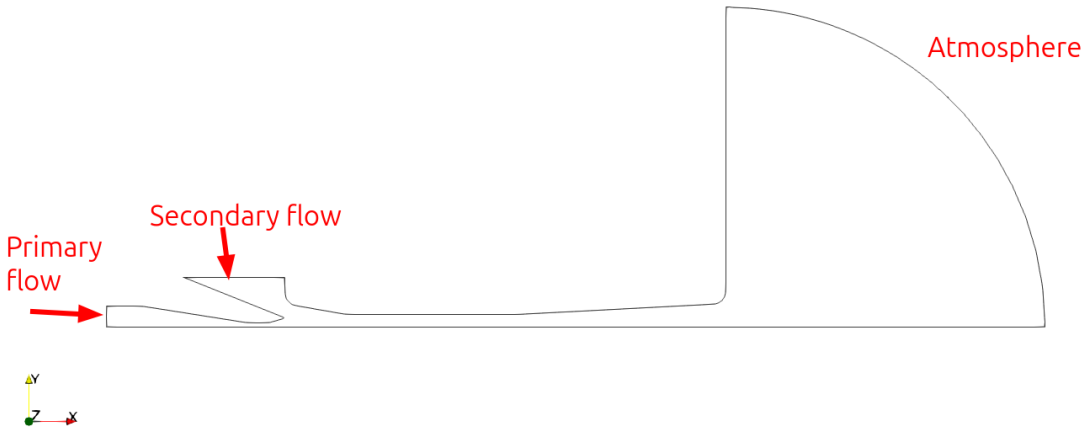

Figure 1: Scheme of supersonic ejector

2 Case Setup

The python environment used in Session 2 has to be also activated for the current tutorial.

3 Part A: Density based explicit solver `rhoCentralFoam`

The clean case can be downloaded from the GitHub repository

3.1 Running the Simulation

Again, the simulation is split in two parts: the generation of the mesh and running the solver.

3.1.1 Mesh generation

To generate the `blockMeshDict` file, check that the python virtual environment is activated and run

```
python supersonic_blockMeshDictGen.py
```

The `blockMeshDict` is created in the same folder. Move it to the `system` folder of the case to be simulated (now the `rCF` case).

The `genMesh.scr` script runs the **`blockMesh`**, **`extrudeMesh`** and **`checkMesh`** commands.

```
./genMesh.scr
```

3.1.2 Initialization and solver execution

To run the solver, type

```
./runCase.scr
```

This simulation is very slow. It takes about 4 to 5 hours to run 5 ms, enough to get a steady state behavior. In order to reduce the runtime for the workshop, a pre-computed case is available in the GitHub repository. Anyway, the clean case can be run for a short time (about 30 minutes) to observe the transient behavior.

3.2 Post-Processing in Terminal

Main and secondary inlets flow rates are visualized with the **`foamMonitor`** utility

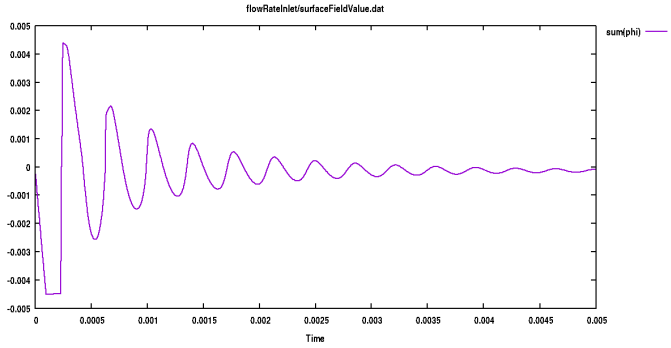

Figure 2: Flow rate evolution for main inlet with **`rhoCentralFoam`** solution.

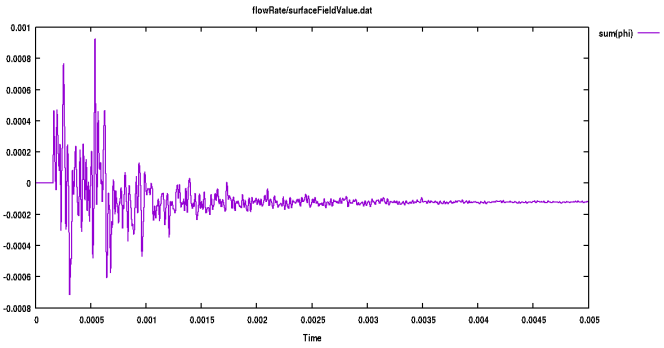

Figure 3: Flow rate evolution in secondary inlet with `rhoCentralFoam` solution.

3.3 Visualization in paraview

Figures 4, 5 and 6 show velocity, Mach number and pressure distribution for the final solution.

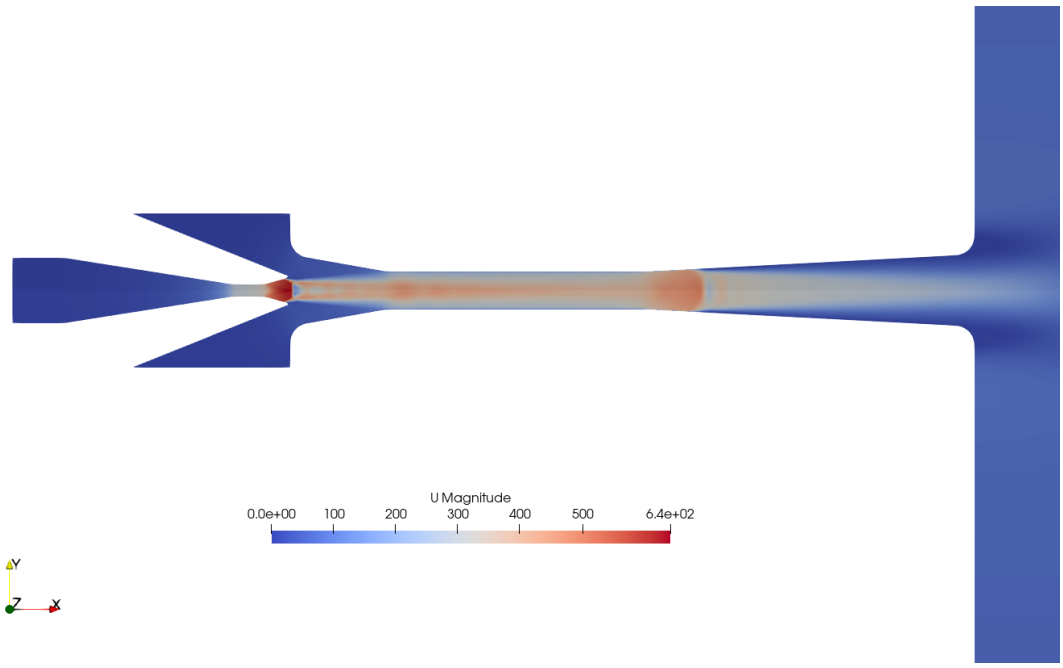

Figure 4: Velocity distribution for `rhoCentralFoam` solution.

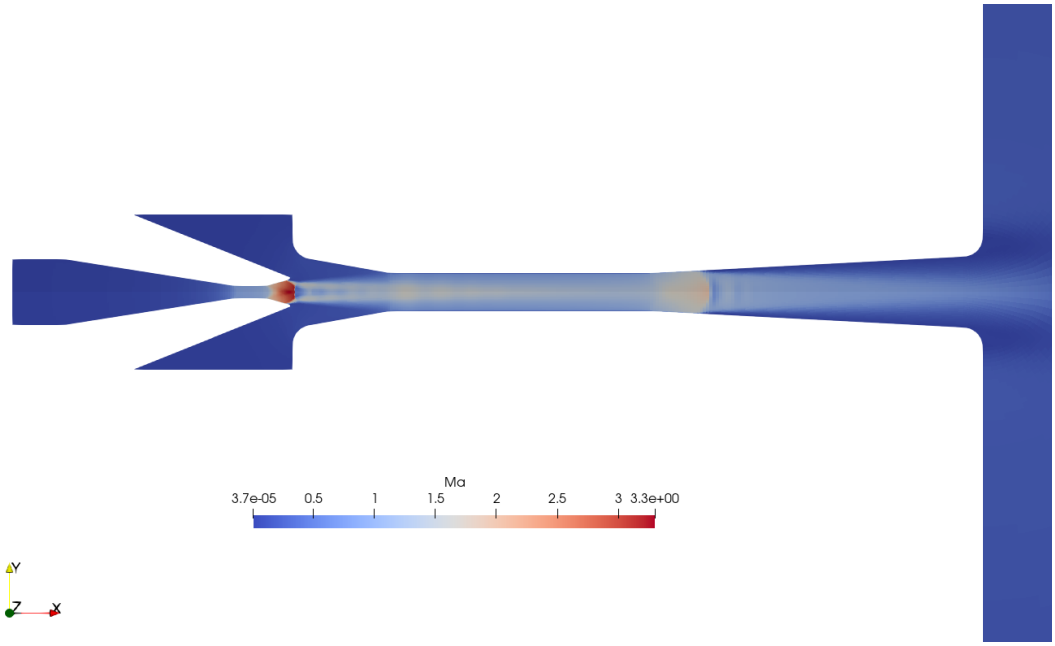

Figure 5: Mach number distribution for `rhoCentralFoam` solution.

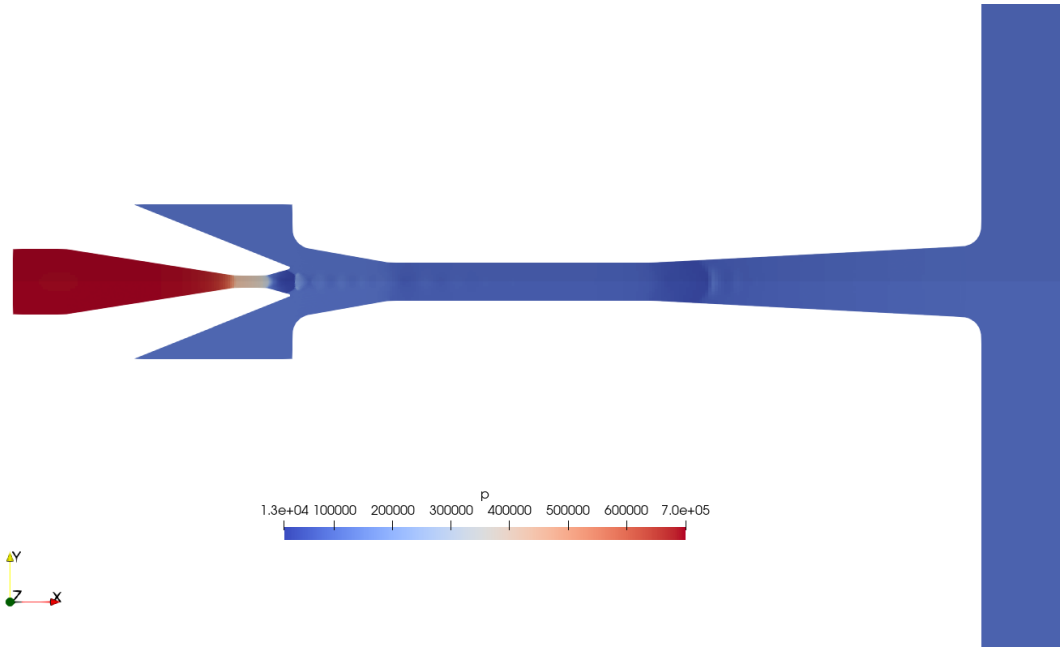

Figure 6: Pressure distribution for `rhoCentralFoam` solution.

Figure 7 shows a plot of the pressure along the x-axis. Shock waves are clearly visible in

this plot.

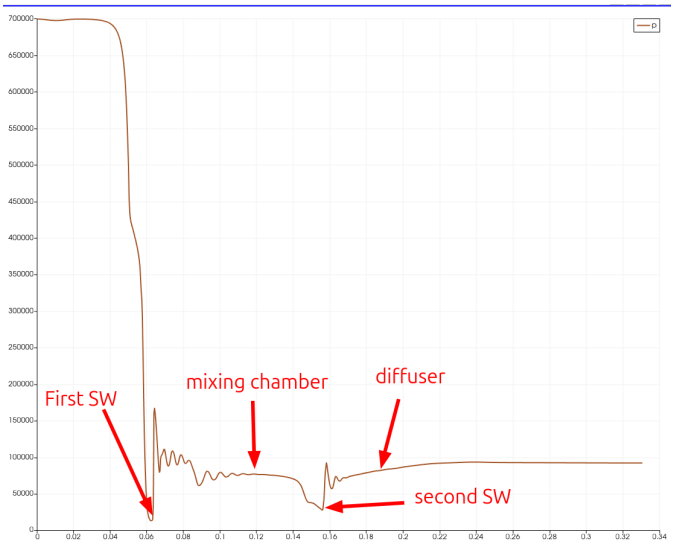

Figure 7: Pressure distribution along x axis for **rhoCentralFoam** solution.

4 Part B: Density based implicit solver HiSA

For this part, for the implicit solver **HiSA** the procedure is basically the same. However, the solver settings, `system/fvSchemes` and `system/fvSolution`, are different because **HiSA** uses a coupled implicit method. The user manual for the solver can be downloaded from the HiSA web page. There is a small typo that has been corrected in the version available in the workshop's GitHub repository.

Firstly, the clean case has to be downloaded from the GitHub repository. Mesh is generated in the same way (actually, it could be used the same mesh of the `rCF` case) and the case is ran with the same script. The computation is faster than with the explicit solver: it takes about 70 minutes to compute 35000 iterations in a Intel Core Ultra 7. Anyway, the case with the solution and postprocessed data is also available in the GitHub repository.

It is important to analyze the special boundary conditions for the **HiSA** case. In order to prevent unstable computation and divergence, characteristic-type boundary conditions are advised to be used. The reference values of velocity, pressure and temperature have to be provided for all fields, and an included file is used in all the files to avoid mistakes

```
...
inletMain
{
  type              characteristicPressureInletOutletVelocity;
  #include          "include/freestreamInletConditions"
```

```
  value           uniform (0 0 0);
}
    outlet
{
  type            characteristicFarfieldVelocity;
  #include        "include/freestreamConditions"
  value           uniform (0 0 0);
}
...
```

where the content of `include/freestreamConditions` is

```
U               (0 0 0);
p               100000;
T               293;
gamma              1.4;
```

and for `include/freestreamInletConditions`

```
U               (0 0 0);
p               700000;
T               293;
gamma              1.4;
```

4.1 Post-Processing in Terminal

Main and secondary inlets flow rates can be plotted again with the **foamMonitor** utility

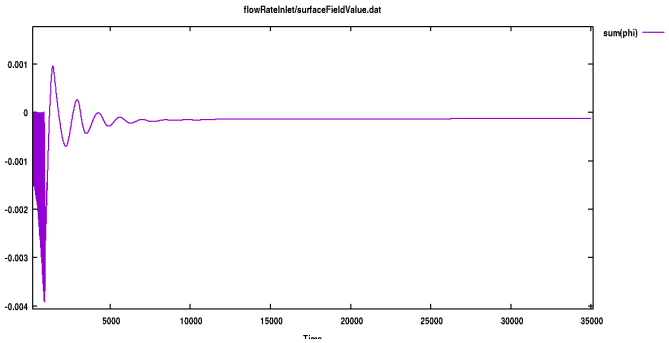

Figure 8: Flow rate evolution for main inlet with **HiSA** solution.

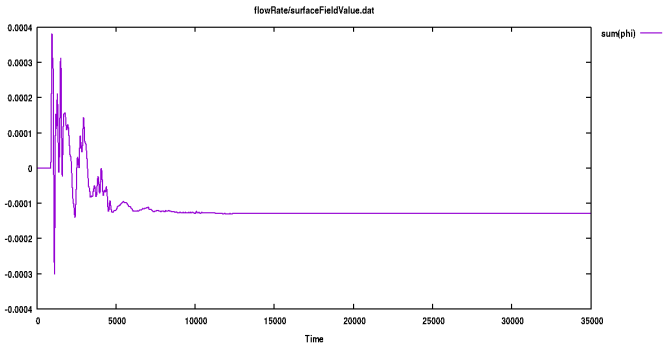

Figure 9: Flow rate evolution for secondary inlet with **HiSA** solution.

4.2 Visualization with paraview

Figures 10, 11 and 12 show velocity, Mach number and pressure distribution for the final solution. Note the similarity with the solution os the **rhoCentralFoam** simulation.

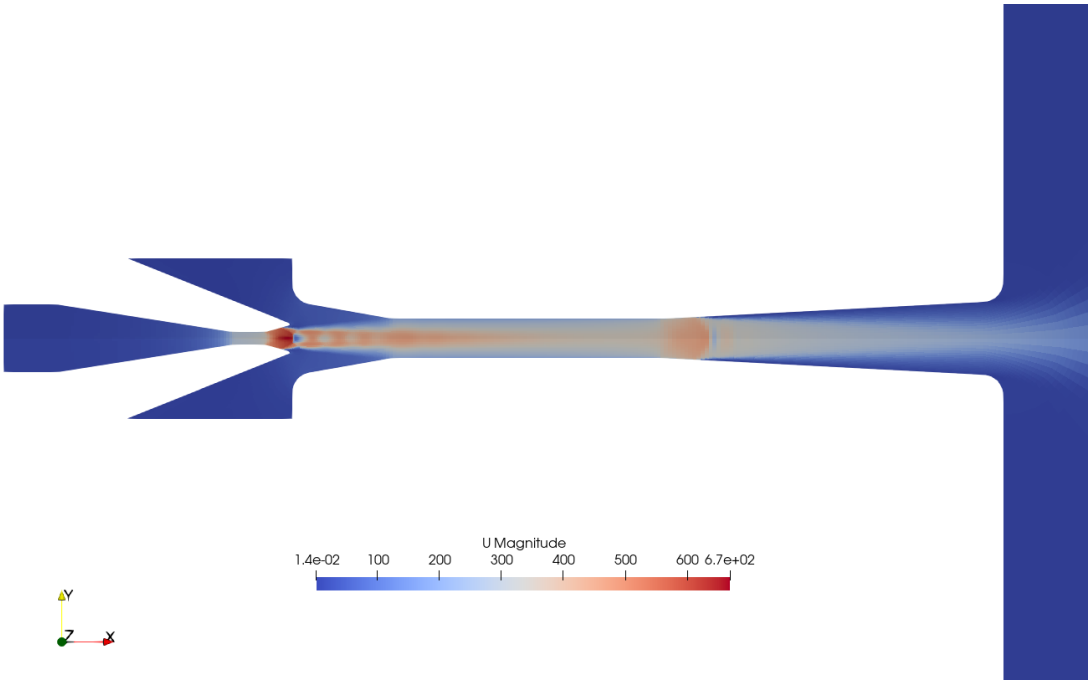

Figure 10: Velocity distribution for **HiSA** solution.

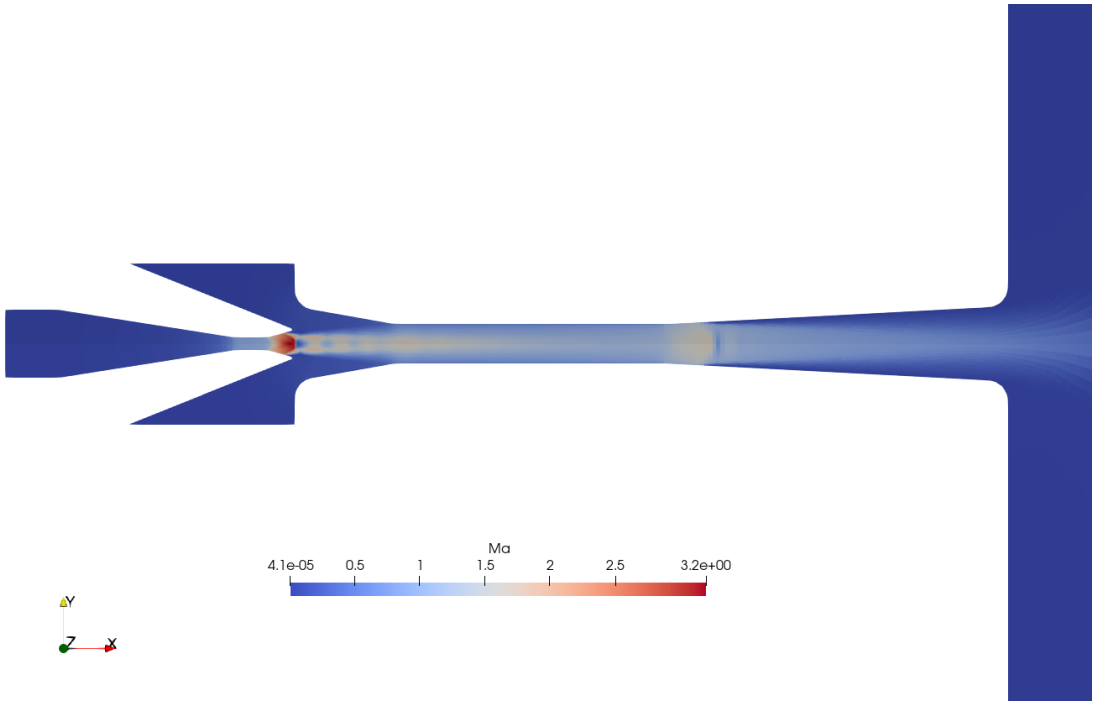

Figure 11: Mach number distribution for `HiSA` solution.

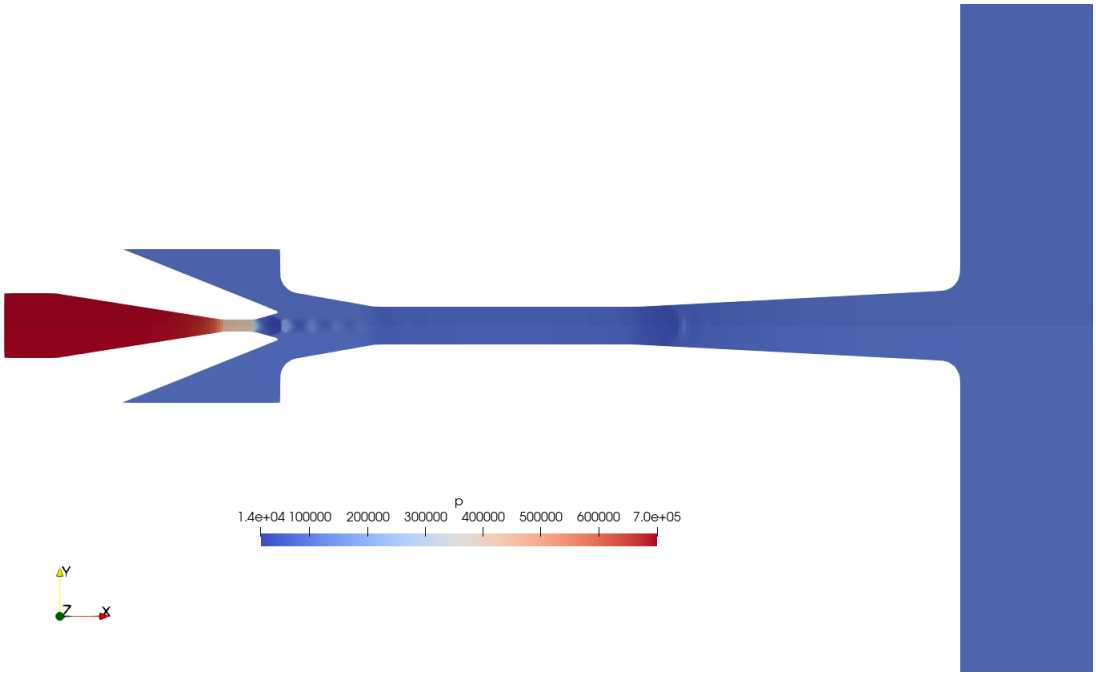

Figure 12: Pressure distribution for `HiSA` solution.

Figure 13 shows the plot of pressure along the x-axis. It is also very similar to the distribution from the explicit solver.

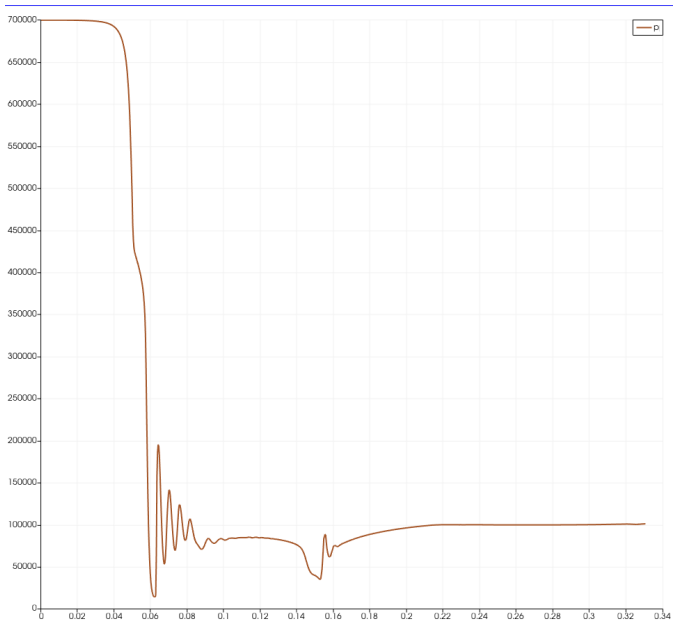

Figure 13: Pressure distribution along x axis for `HiSA` solution.

5 Student Exercises and Extensions

1. Along with flow rates in inlet, the cases also provides the flow rate in outlet. Note that the flow rate is not balance for a considerable error. Why is that? How can it be corrected?

2. For atmospheric pressure in secondary inlet the second shock wave is generated at the beginning of the diffuser. However, when secondary pressure decreases, this shock wave goes back in the mixing chamber, as is shown in Figure 11 of Macià's paper, reproduced in here in Figure 14.

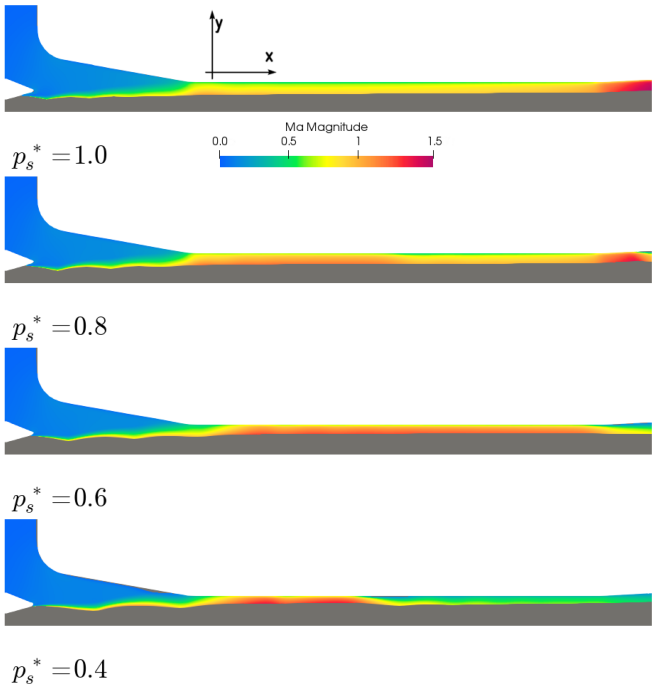

Figure 14: Mach number distribution for several secondary pressure, as published in the paper by Macià et al.

Check this behavior by changing secondary pressure to 0.5 bar. If the flow starts from the solution for 1 bar, it will converge faster.

3. Although solutions for both explicit and implicit solvers are similar, the performance is different in function of the secondary pressure, as shown in Figure 10 of Macià's paper, reproduced here as Figure 15.

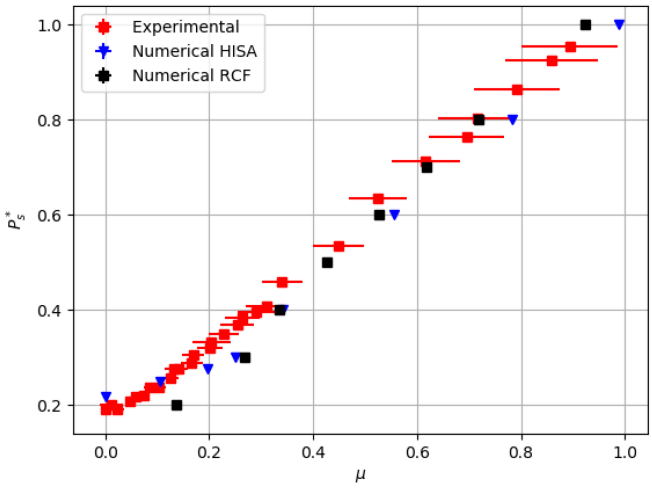

Figure 15: Comparison of experimental measurments and CFD simulations with explicit and implicit solvers.

Note that the `HiSA` solver gives better results for low secondary pressure. It is a good exercise to "close" the secondary inlet (define it as a `wall`) and see the results from both solvers. The average pressure has to be monitored in this patch.

Disclaimer

This tutorial has been developed by **Robert Castilla**, of the Departament of Fluid Mechanics in the Universitat Politècnica de Catalunya, for the doctoral workshop at **IMPPAN Gdańsk** (July 2026). The author utilized **DeepSeek** for the initial definition of the document format. Further technical refinement and the development of advanced post-processing exercises were supported by **NotebookLM**. All the technical information for the **OpenFOAM v2512** environmnet and all physical interpretations were performed and approved by the author.